

บทที่ 9 : การบริหาร โครงการซอฟต์แวร์

แนะนำการบริหารโครงการซอฟต์แวร์ **01**

ขั้นตอนของการบริหารโครงการ **02**

การประมาณค่าความพยายามซอฟต์แวร์ **03**

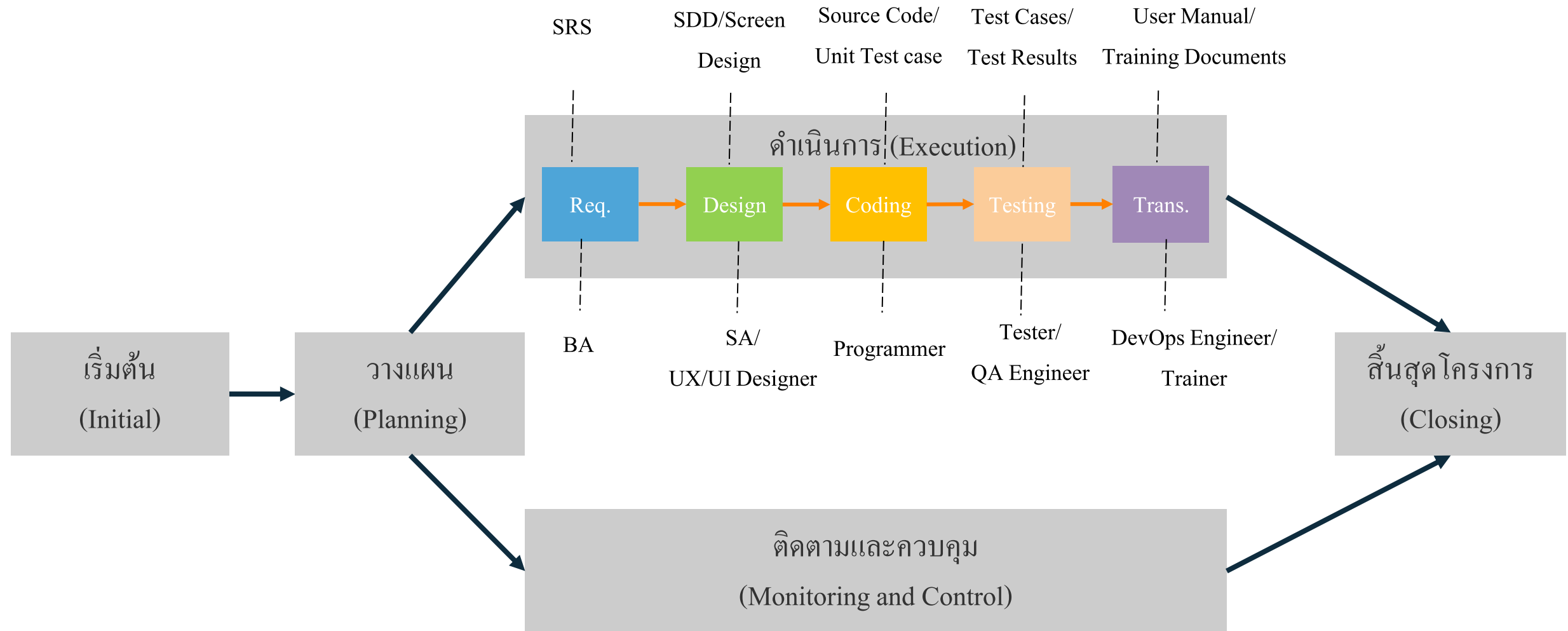
1. แนะนำการบริหารโครงการซอฟต์แวร์

โครงการ (Project) คือ การดำเนินการชั่วคราว ซึ่งสัญญาว่าจะสร้างผลิตภัณฑ์หรือบริการให้แล้วเสร็จ ภายใต้วัตถุประสงค์ (Objective) ขอบเขตงาน (Scope) เวลา (Time) และทรัพยากร (Cost/Staff) ที่กำหนด

โครงการซอฟต์แวร์ (Software Project) คือ การดำเนินการชั่วคราว ซึ่งสัญญาว่าจะสร้าง**ซอฟต์แวร์**ให้แล้วเสร็จ ภายใต้วัตถุประสงค์ ขอบเขตงาน เวลา และทรัพยากรที่กำหนด

การบริหารโครงการ (Project Management) คือ การประยุกต์ใช้องค์ความรู้ ทักษะ เครื่องมือ และเทคนิค เพื่อดำเนินกิจกรรมของโครงการ ให้บรรลุวัตถุประสงค์ของโครงการ

การบริหารโครงการซอฟต์แวร์ (Software Project Management) คือ การประยุกต์ใช้องค์ความรู้ ทักษะ เครื่องมือ และเทคนิค เพื่อดำเนินกิจกรรมของโครงการ**ซอฟต์แวร์** ให้บรรลุวัตถุประสงค์ของโครงการ



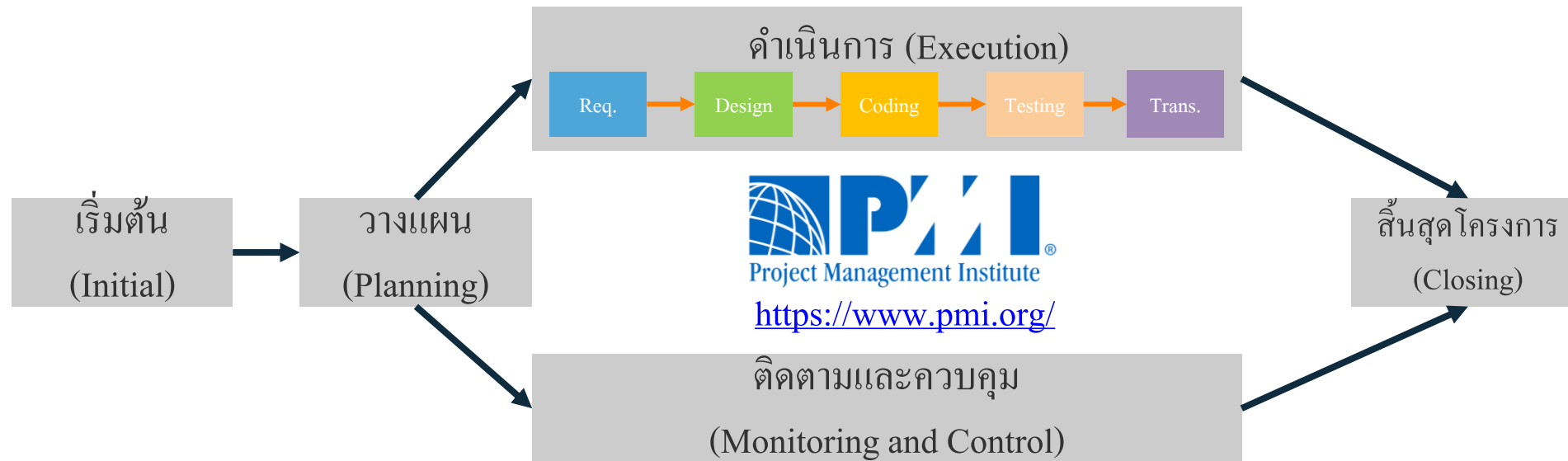
ภาพ : การบริหาร โครงการและกระบวนการพัฒนาซอฟต์แวร์

❑ ปัจจัยที่ส่งผลกระทบต่อการบริหารโครงการซอฟต์แวร์

- **ประเภทของซอฟต์แวร์ :** ซอฟต์แวร์ที่มีผลกระทบกับชีวิตหรือเงินจำนวนมาก ควรเน้นในเรื่องความปลอดภัยของซอฟต์แวร์ (Security Management) และการบริหารความเสี่ยง (Risk Management)
- **ขนาดของซอฟต์แวร์ :** ขนาดซอฟต์แวร์ใหญ่ทำให้ยากต่อการบริหารอาจเน้นการบริหารความเสี่ยง (Risk Management)
- **ลูกค้า/ผู้ใช้งาน/ทีมงาน :** ลูกค้า/ผู้ใช้งาน/ทีมงานที่เกี่ยวข้องไม่ค่อยร่วมมือกันทำงาน อาจะเน้นในการบริหารการสื่อสาร (Communication Management) และการบริหารผู้มีส่วนได้ส่วนเสีย (Stakeholder Management)
- **กระบวนการพัฒนาซอฟต์แวร์ :**
 - ถ้าดำเนินการแบบ Waterfall อาจะเน้นในการวางแผนและการส่งมอบเอกสาร
 - ถ้าดำเนินการแบบ Agile อาจะเน้นในการบริหารการเปลี่ยนแปลง (Change Management)

2. ขั้นตอนของการบริหารโครงการ

- 1 ขั้นเริ่มต้นโครงการ (Initiating) : ศึกษา Requirements ระดับ High-Level และศึกษาความเป็นไปได้
- 2 ขั้นวางแผนโครงการ (Planning) : กำหนด Scope, Time, Staff, Cost, และ Quality
จัดทำ Risk plan และ Communication plan
- 3 ขั้นดำเนินการ (Executing) : ดำเนินงานตาม Plan
- 4 ขั้นติดตามและควบคุม (Monitoring and Controlling) : ติดตามการดำเนินงาน SDLC ว่าเป็นไปตาม Scope, Time, Staff, Cost, และ Quality ที่กำหนดไว้หรือไม่
- 5 ขั้นสิ้นสุดโครงการ (Closing) : สรุปกำไร/ขาดทุน, สรุปสิ่งที่ได้เรียนรู้จากโครงการ, ส่งต่องานให้ทีม support



3. การประเมินค่าความพยายามซอฟต์แวร์

- เป็นกิจกรรมที่สำคัญในการบริหารจัดการ โครงการซอฟต์แวร์
- ใช้สำหรับการประมาณระยะเวลา จำนวนคน และต้นทุน
- ถ้าการประมาณการสูงกว่าความเป็นจริง (Over-estimate) แล้วมีโอกาสพลาดการได้รับงานประมูล
- ถ้าการประมาณการต่ำกว่าความเป็นจริง (Under-estimate) โครงการมีโอกาสเสร็จไม่ทันตามกำหนด หรือขาดทุน

❑ ประเภทการประมาณค่าความพยายามซอฟต์แวร์ (Software Effort Estimation)

- การประมาณการโดยผู้เชี่ยวชาญ (Expert Judgment) : เป็นการประมาณการ โดยใช้กระบวนการของผู้เชี่ยวชาญ
 - Delphi
 - Wideband Delphi
 - Planning Poker
- การประมาณการด้วยแบบจำลอง (Formal Estimation Model) : เป็นการประมาณการ โดยใช้แบบจำลองคณิตศาสตร์
 - Parametric Model เช่น COCOMO, SLIM, and SEER for Software
 - Machine Learning เช่น ANN, SVM, Logit, and KNN
- การประมาณการบนพื้นฐานของการรวมกัน (Combination-based Estimation) : เป็นการประมาณการจากผลที่ได้จากผู้เชี่ยวชาญและแบบจำลองการประมาณการ
 - การหาค่าเฉลี่ย (Average)
 - การหาค่าเฉลี่ยโดยการถ่วงน้ำหนัก (Weighted Average)

□ การประมาณแบบ Delphi

ขั้นที่ 1 เลือกผู้เชี่ยวชาญ (Experts) และผู้ควบคุมการประมาณ (Moderator)

ขั้นที่ 2 ผู้เชี่ยวชาญแต่ละคนทำการประมาณด้วยตนเอง จากคำอธิบายและขอบเขตโครงการ (ผู้เชี่ยวชาญแต่ละคนจะไม่รู้ว่าใครให้ค่าประมาณการเท่าไร)

ขั้นที่ 3 ผู้ควบคุมรวบรวมผลประมาณการและสรุปผล

ขั้นที่ 4 ผู้ควบคุมรายงานผลสรุปให้กับผู้เชี่ยวชาญแต่ละคน

- ถ้าผลการประมาณตรงกันหรือใกล้เคียงกันให้ ให้จบกระบวนการประมาณ
- ถ้ายังไม่ใกล้เคียงกัน ผู้ควบคุมจะให้ข้อมูลเกี่ยวกับโครงการเพิ่มเติม แล้วดำเนินการในขั้นตอนที่ 2 ไปจนกว่าผลการประมาณตรงกันหรือใกล้เคียงกัน

❑ การประมาณแบบ Wideband Delphi

- พัฒนาโดย Barry Boehm and Farquhar
- ปรับปรุงจากวิธี Delphi โดยให้กลุ่มผู้เชี่ยวชาญสามารถปรึกษากันได้

ขั้นที่ 1 เลือกผู้เชี่ยวชาญ (Experts) และผู้ควบคุมการประมาณ (Moderator)

ขั้นที่ 2 ผู้ควบคุมเตรียมงานและนำผู้เชี่ยวชาญทำการประชุม เพื่อหาข้อสรุปของขอบเขตโครงการ
โดยการทำ WBS

ขั้นที่ 3 ผู้เชี่ยวชาญแต่ละคนทำการประมาณการค่าความพยายามในแต่ละงาน (Task) จาก WBS

ขั้นที่ 4 ประชุมเพื่อหาข้อสรุปของผลประมาณการร่วมกันระหว่างผู้เชี่ยวชาญ

ขั้นที่ 5 ผู้ควบคุมรวบรวมผลการประมาณการและหารือกับผู้เชี่ยวชาญ ถ้าผลการประเมินเป็นที่พอใจให้
จบกระบวนการ แต่ถ้าไม่ใช่ให้ไปทำขั้นตอนที่ 3

❑ การประมาณแบบ Planning Poker

- พัฒนาโดย James Grenning ปี 2002
- มักใช้ใน Agile Development เช่น Scrum, Extreme Programming
- เปลี่ยนจากการพูดคุยเป็นการเปิดป้ายคะแนน (Card)

ขั้นที่ 1 Product Owner ให้คำอธิบายโดยรวมของ 1 User Story ทีมพัฒนาสอบถามข้อสงสัย แล้วทำการปรึกษาหารือกัน ผู้ควบคุม (Moderator) ทำการบันทึกผลสรุป

ขั้นที่ 2 แต่ละคนใช้เลือกป้ายคะแนน แล้วคว่ำป้ายลงบนโต๊ะ

ขั้นที่ 3 ทุกคนเปิดป้ายคะแนนพร้อมกัน

ขั้นที่ 4 คนที่ให้คะแนนสูงสุดและต่ำสุด ทำการอธิบายถึงที่มาของคะแนน

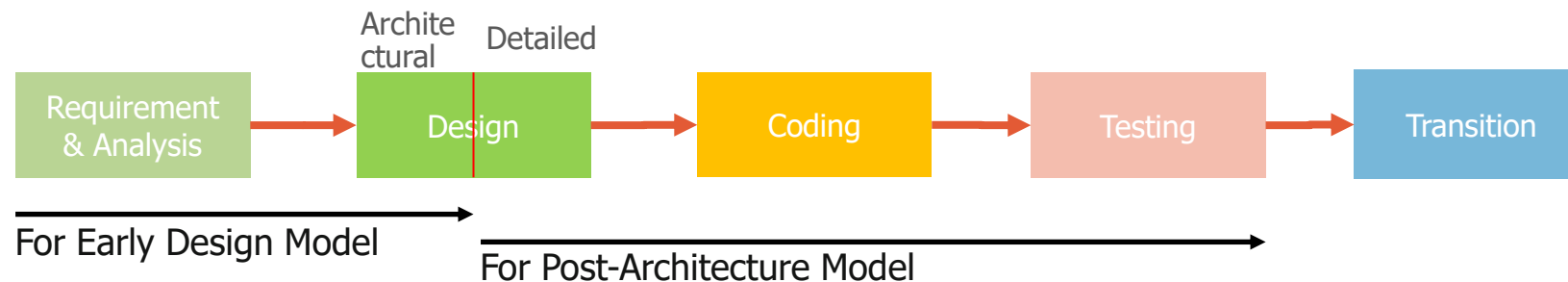
ขั้นที่ 5 ย่อยกลับไปทำข้อ 2 จนกว่าจะได้ข้อสรุปร่วมกัน

❑ การประมาณแบบ COCOMO Model

- COCOMO มาจาก The COConstructive COst MOdel
- เป็น Parametric Model
- ใช้ SLOC เป็นปัจจัยหลักในการคำนวณ
- หากมีการปรับปรุง COCOMO ใหม่ด้วยข้อมูลองครค์เอง โดยการทำให้การประมาณการมีความแม่นยำสูงขึ้น
- COCOMO ประกอบไปด้วย
 - COCOMO 81 หรือ COCOMO I พัฒนาโดย Barry Boehm ปี 1981 ณ University of Southern California ประกอบไปด้วย
 - Basic : ง่ายเนื่องจากใช้เฉพาะ SLOC สำหรับการคำนวณ แต่ให้ความถูกต้องแบบหยาบ ๆ
 - Intermediate : มีการเพิ่มปัจจัยอื่น ๆ สำหรับการประมาณการ
 - Detailed : มีการประมาณการแยกในแต่ละขั้นตอนของการพัฒนาซอฟต์แวร์ (Phase)
 - COCOMO II พัฒนาในปี 1995 แต่เผยแพร่ ปี 2000 ในหนังสือชื่อ “Software Cost Estimation with COCOMO II”

□ องค์ประกอบของ COCOMO II

- COCOMO II ประกอบไปด้วย
- Application Composition :
 - เหมาะสำหรับซอฟต์แวร์ที่พัฒนาด้วย Modern GUI-Builder Tools
 - ใช้ Application Point เป็นปัจจัยในการคำนวณ
- Early Design :
 - เหมาะสำหรับการประมาณในช่วงต้นการพัฒนาจนถึงขั้น Architectural Design
 - ใช้ SLOC เป็นปัจจัยหลักและปัจจัยอื่น ๆ อีกเพียง 13 (5+7) ปัจจัย เนื่องจากยังไม่ทราบรายละเอียดของปัจจัยที่เกี่ยวข้องมากนัก
- Post-Architecture
 - เหมาะสำหรับการประมาณตั้งแต่ Detailed Design
 - ใช้ SLOC เป็นปัจจัยหลัก ร่วมกับปัจจัยอื่น ๆ อีก 22 (5+17) ปัจจัย



สูตรการคำนวณ

$$PM = Ax(Size)^E \prod_{i=1}^{17} EM_i$$

$$E = B + 0.01 \sum_{i=1}^5 SF_i$$

ตัวแปร	คำอธิบาย
PM	ค่าความพยายามที่คำนวณได้ มีหน่วยเป็น Person-month
A	ค่าคงที่ ซึ่งมีค่า Default เป็น 2.94
Size	ขนาดของซอฟต์แวร์ ในหน่วย Kilo Source Line Of Code (KSLOC)
E	ค่า Scale Exponent
B	ค่าคงที่ ซึ่งมีค่า Default เป็น 0.91
SF	ค่า Scale Factor เป็นค่าที่แสดงถึงคุณลักษณะของการพัฒนาซอฟต์แวร์ ซึ่งมีผลกระทบต่อค่า Effort ในอัตราที่เพิ่มขึ้นหรือลดลงแบบ Exponential เช่น ค่า precedentedness, process maturity.
EM	Effort Multiplier เป็นค่าที่แสดงถึงปัจจัย (Effort Driver) ที่ส่งผลกระทบค่าประมาณการ Effort ซึ่งแบ่ง Effort Driver ออกเป็น 4 กลุ่ม หรือ 4 P ประกอบด้วย Product, Project, Personel, และ Platform



Scale Factors (SF)	ระดับคะแนน (Rating)					
	Very Low	Low	Nominal	High	Very High	Extra High
ความคุ้นเคยกับระบบ (Precedentedness:PREC)	6.20	4.96	3.72	2.48	1.24	0.00
ความยืดหยุ่นในการพัฒนา (Development Flexibility : FLEX)	5.07	4.05	3.04	2.03	1.01	0.00
การจัดการความเสี่ยง (Risk Resolution : RESL)	7.07	5.65	4.24	2.83	1.41	0.00
ความสัมพันธ์ภายในทีม (Team Cohesion : TEAM)	5.48	4.38	3.29	2.19	1.10	0.00
ระดับของกระบวนการ (Process Maturity : PMAT)	7.80	6.24	4.68	3.12	1.56	0.00

Effort Drivers (EM)	ระดับคะแนน (Rating)					
	Very Low	Low	Nominal	High	Very High	Extra High
ด้านผลิตภัณฑ์ (Product attributes)						
Required Software Reliability (RELY)	0.82	0.92	1.00	1.10	1.26	-
Data Base Size (DATA)	-	0.90	1.00	1.14	1.28	-
Product Complexity (CPLX)	0.73	0.87	1.00	1.17	1.34	1.74
Developed for Reusability (RUSE)	-	0.95	1.00	1.07	1.15	1.24
Documentation Match to Lifecycle Needs (DOCU)	0.81	0.91	1.00	1.11	1.23	-
ด้านแพลตฟอร์ม (Platform attributes)						
Execution Time Constraint (TIME)	-	-	1.00	1.11	1.29	1.63
Main Storage Constraint (STOR)	-	-	1.00	1.05	1.17	1.46
Platform Volatility (PVOL)	-	0.87	1.00	1.15	1.30	-
ด้านทีมงาน (Personnel attributes)						
Analyst Capability (ACAP)	1.42	1.19	1.00	0.85	0.71	-
Programmer Capability (PCAP)	1.34	1.15	1.0	0.88	0.76	-
Personnel Continuity (PCON)	1.29	1.12	1.00	0.90	0.81	-
Application Experience (AEXP)	1.22	1.10	1.00	0.88	0.81	-
Platform Experience (PLEX)	1.19	1.09	1.00	0.91	0.85	-
Language and Tool Experience (LTEX)	1.20	1.09	1.00	0.91	0.84	-
ด้านโครงการ (Project attributes)						
Use of Software Tools (TOOL)	1.17	1.09	1.00	0.90	0.78	-
Multisite Development (SITE)	1.22	1.09	1.00	0.93	0.86	0.80
Required Development Schedule (SCED)	1.43	1.14	1.00	1.00	1.00	-

❑ การวัด Lines of Code (Lines of Code Measurement)

Criteria	Physical	Logical
Executable Line	นับ	นับ
Compiler Directive/Declaration Line	นับ	นับ
Comment Line	ไม่นับ	ไม่นับ
Blank Line	ไม่นับ	ไม่นับ
ตัวอย่างการนับ	// Compute Grade If (score >= 80) { print "A"; }	// Compute Grade If (score >= 80) { print "A"; }
ผลการนับ	LOC = 3	LOC = 2

❑ การวัด Function Points

- Function Points (FP) คือ ผลที่ได้จากการวัดขนาดซอฟต์แวร์เชิงฟังก์ชัน ซึ่งวัดฟังก์ชันการทำงานของซอฟต์แวร์ในมุมมองของ User
- FP สามารถคำนวณได้ ณ ช่วงต้นของการพัฒนาซอฟต์แวร์ ซึ่งทำให้สามารถนำไปใช้สำหรับการคำนวณ Effort/Cost เพื่อใช้ในการวางแผนโครงการ
- Function Points Analysis (FPA) ถือเป็นวิธีแรกของการวัดขนาดซอฟต์แวร์เชิงฟังก์ชัน (นั่นคือ FP) ซึ่งเป็นพื้นฐานของการวัดขนาดซอฟต์แวร์เชิงฟังก์ชันอื่น ๆ FPA ถูกเผยแพร่ครั้งแรกในปี 1979 โดย Arbrecht
- ในปัจจุบันมีหลายวิธีในการวัด FP และหลาย ๆ วิธีเป็นวิธีวัดที่เป็นมาตรฐาน ISO เช่น
 - IFPUG Functional Size Measurement Method
 - Mk II Function Point Analysis
 - NESMA Functional Size Measurement Method
 - FisMA Functional Size Measurement Method
 - COSMIC Functional Size Measurement Method

❑ การวัด Function Points

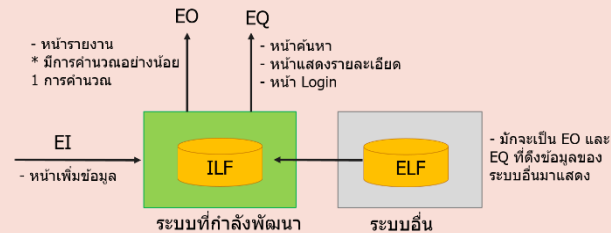
- Function Points (FP) คือ ผลที่ได้จากการวัดขนาดซอฟต์แวร์เชิงฟังก์ชัน ซึ่งวัดฟังก์ชันการทำงานของซอฟต์แวร์ในมุมมองของ User
- FP สามารถคำนวณได้ ณ ช่วงต้นของการพัฒนาซอฟต์แวร์ ซึ่งทำให้สามารถนำไปใช้สำหรับการคำนวณ Effort/Cost เพื่อใช้ในการวางแผนโครงการ
- Function Points Analysis (FPA) ถือเป็นวิธีแรกของการวัดขนาดซอฟต์แวร์เชิงฟังก์ชัน (นั่นคือ FP) ซึ่งเป็นพื้นฐานของการวัดขนาดซอฟต์แวร์เชิงฟังก์ชันอื่น ๆ FPA ถูกเผยแพร่ครั้งแรกในปี 1979 โดย Arbrecht
- ในปัจจุบันมีหลายวิธีในการวัด FP และหลาย ๆ วิธีเป็นวิธีวัดที่เป็นมาตรฐาน ISO เช่น
 - IFPUG Functional Size Measurement Method
 - Mk II Function Point Analysis
 - NESMA Functional Size Measurement Method
 - FisMA Functional Size Measurement Method
 - COSMIC Functional Size Measurement Method

IFPUG Functional Size Measurement Method

- ❑ IFPUG เป็นวิธีวัดที่พัฒนาต่อยอดจาก FPA โดยตรง
- ❑ พัฒนาและดูแลโดย International Function Point User Group (IFPUG)
- ❑ การคำนวณ Function Points (FP) แบ่งออกเป็น 2 ส่วน

ส่วนที่ 1 ประเภทฟังก์ชัน (Function Type) ของระบบ ในมุมมองของผู้ใช้งาน
ต่อการประมวลผลข้อมูล ประกอบด้วย

1. External Input (EI)
2. External Output (EO)
3. External Inquiry (EQ)
4. Internal Logical File (ILF)
5. External Logical File (ELF)



ส่วนที่ 2 คุณลักษณะทั่วไปของระบบ (General System Characteristics) 14 ข้อ
แต่ละข้อมีอันดับของคะแนนตั้งแต่ 0 - 5 โดยที่
0 หมายถึงคุณลักษณะที่ว่ามีผลกระทบต่อระบบน้อยที่สุด
5 หมายถึงคุณลักษณะที่ว่ามีผลกระทบต่อระบบมากที่สุด

สูตรการคำนวณ Function Point ของ IFPUG

ส่วนที่ 1

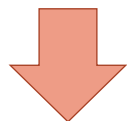
UFP = ฟังก์ชันพอยต์ก่อนปรับคะแนน (Unadjusted Function Point)

ส่วนที่ 2

TDI = ผลรวมคะแนนจากคุณลักษณะทั่วไปของระบบ (Total Degree of Influence)

VAF = คะแนนสำหรับการปรับค่าของฟังก์ชันพอยต์ (Value Adjustment Factor)

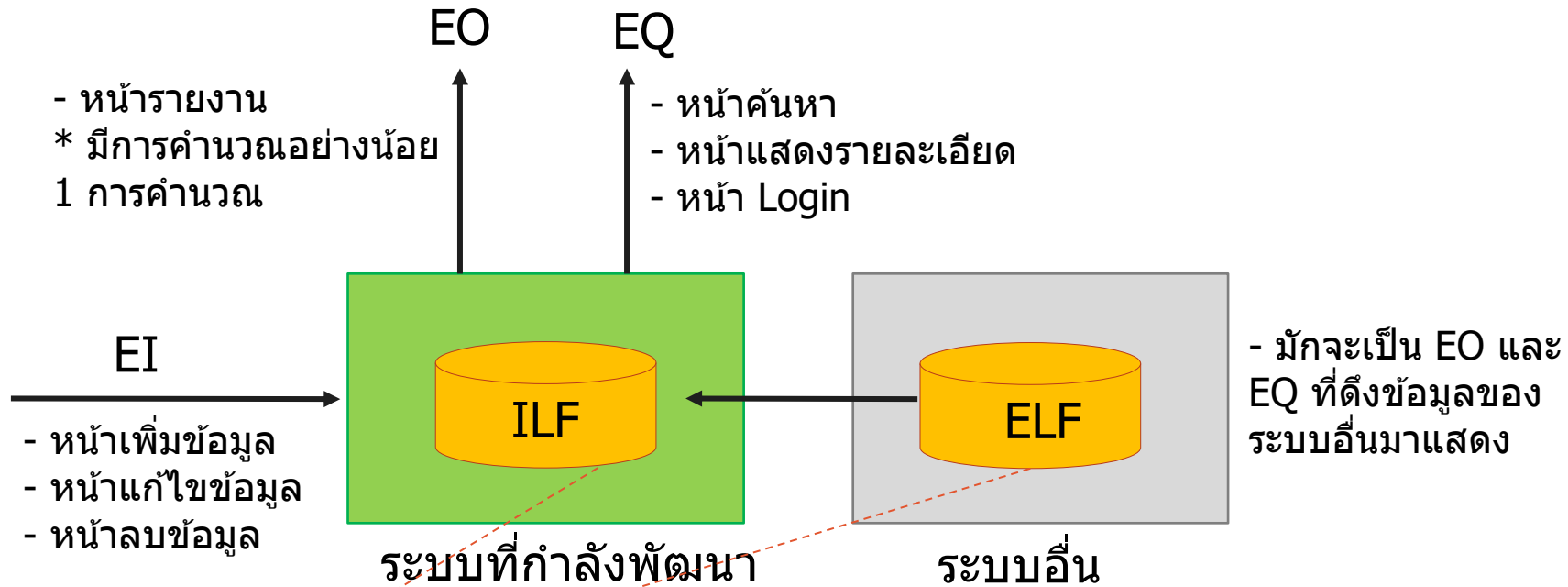
$$= (TDI * 0.01) + 0.65$$



AFP = ฟังก์ชันพอยต์หลังปรับคะแนน (Adjusted Function Point)

$$= UFP * VAF$$

ส่วนที่ 1 : ประเภทฟังก์ชัน (Function Type)



รูปแสดงส่วนประกอบของประเภทฟังก์ชัน ของ IFPUG

- ILF และ ELF เป็นข้อมูลในระดับ Logical นั่นคือ ข้อมูลที่อยู่ระหว่าง 1-2 NF
- ตัวอย่าง ILF เช่น ข้อมูลพนักงาน ข้อมูลลูกค้า ข้อมูลสินค้า ข้อมูลสถานที่จัดเก็บสินค้า (โกดัง) และข้อมูลการสั่งซื้อ
- ตัวอย่างที่ไม่นับเป็น ILF เช่น ข้อมูลรายละเอียดการสั่งซื้อ (ซึ่งประกอบด้วย รหัสการสั่งซื้อ รหัสสินค้า ราคา และจำนวน) ให้ถือเป็นข้อมูลการสั่งซื้อ แต่นับเป็น 2 RET และนับ DET รวมกัน

รายการ	ไฟล์ข้อมูลเชิงตรรกะ (Logical File)	Function Type		RET	DET	Complexity
		ILF	EIF			
1	ข้อมูลพนักงาน					
2	ข้อมูลลูกค้า					
3	ข้อมูลสินค้า					
4	ข้อมูลสถานที่จัดเก็บสินค้า					
5	ข้อมูลการสั่งซื้อสินค้า					
...						
...						
...						
...						
...						
...						
...						
...						
...						
...						
...						

ตารางแสดง แบบฟอร์มการกำหนดความซับซ้อนของไฟล์ข้อมูล

- Record Element Type (RET)

- นับกลุ่มย่อยของฟิลด์ข้อมูลใน ILF/EIF ที่ผู้ใช้งานสามารถรับรู้ได้ (1 กลุ่ม = 1 RET)
- โดยปกติ 1 ILF/EIF จะมี 2 RET
- ตัวอย่างการนับ RET เช่น
 - ข้อมูลการสั่งซื้อกับรายละเอียดการซื้อขาย นับเป็น 2 RET และ 1 ILF/EIF
 - ข้อมูลพนักงานกับผู้ที่ได้รับการอุปการะและประวัติการศึกษา นับเป็น 3 RET และ 1 ILF/EIF
 - ข้อมูล Job กับ Job Description นับเป็น 2 RET และ 1 ILF/EIF

- Data Element Type (DET)

- นับฟิลด์ข้อมูลที่ผู้ใช้งานสามารถรับรู้ได้ เช่น ชื่อสินค้า รายละเอียดสินค้า (1 ฟิลด์ = 1 RET)
- ไม่นับฟิลด์ข้อมูลที่ผู้ใช้งานไม่จำเป็นต้องระบุหรือทราบ เช่น รหัสสินค้าที่เป็น Auto Running Number
- นับปุ่มบันทึก เป็น 1 RET
- นับ Error Message, Confirm Message, หรือ Success Message เป็นเพียงแค่ 1 RET

รายการ	รายการฟังก์ชัน (Transactional Function)	Function Type			FTR	DET	Complexity
		EI	EO	EQ			
1	เข้าสู่ระบบ						
	การจัดการข้อมูลพนักงาน						
2	เพิ่มข้อมูลพนักงาน						
3	ลบข้อมูลพนักงาน						
4	แก้ไขข้อมูลพนักงาน						
5	แสดงรายละเอียดข้อมูลพนักงาน						
6	ค้นหาและแสดงรายการพนักงาน						
7	รายงานสรุปข้อมูลพนักงาน						
	การจัดการข้อมูลสินค้า						
8	เพิ่มข้อมูลสินค้า						
9	ลบข้อมูลสินค้า						
10	แก้ไขข้อมูลสินค้า						
11	แสดงรายละเอียดข้อมูลสินค้า						
12	ค้นหาและแสดงรายการสินค้า						
...							
...							

ตารางแสดง แบบฟอร์มการกำหนดความซับซ้อนของฟังก์ชัน

- **File Type Referenced (FTR)** อ้างถึง ILF/EIF ที่ถูกอ่านหรือนำเข้าข้อมูล ผ่าน EI/EO/EQ

ILF and EIF Functional Complexity

		DETs		
		1 – 19	20 – 50	> 50
RETs	1	Low	Low	Average
	2 – 5	Low	Average	High
	> 5	Average	High	High

EI Functional Complexity

		DETs		
		1 – 4	5 – 15	> 15
FTRs	0 – 1	Low	Low	Average
	2	Low	Average	High
	> 2	Average	High	High

EO and EQ Functional Complexity

		DETs		
		1 – 5	6 – 19	> 19
FTRs	0 – 1	Low	Low	Average
	2 – 3	Low	Average	High
	> 3	Average	High	High
NOTE		An EQ has a minimum of 1 FTR.		

ตารางแสดงการกำหนดความซับซ้อนของ
ประเภทฟังก์ชัน

WEIGHTED COMPLEXITY OF FUNCTIONS

	Low	Average	High
ILF	7	10	15
EIF	5	7	10
EI	3	4	6
EO	4	5	7
EQ	3	4	6

ตารางแสดงน้ำหนักของความซับซ้อนของ
ประเภทฟังก์ชัน

Function Type	Complexity			ผลรวม
	Low	Average	High	
ILF	_____ x 7 = _____	_____ x 10 = _____	_____ x 15 = _____	
ELF	_____ x 5 = _____	_____ x 7 = _____	_____ x 10 = _____	
EI	_____ x 3 = _____	_____ x 4 = _____	_____ x 6 = _____	
EO	_____ x 4 = _____	_____ x 5 = _____	_____ x 7 = _____	
EQ	_____ x 3 = _____	_____ x 4 = _____	_____ x 6 = _____	
UFP				

ตารางแสดง แบบฟอร์มการคำนวณ UFP

ส่วนที่ 2 : คุณลักษณะทั่วไปของระบบ (General System Characteristics)

รายการ	คุณลักษณะทั่วไปของระบบ (General Systems Characteristics)	แนวทางการใช้คะแนน (Hint)	เหตุผล	คะแนน (0-5) (Degree of Influence)
1	การสื่อสารข้อมูล (Data Communication)	0-3:Batch; 4:Online; 4-5:Web/Realtime (RT)		
2	การประมวลผลข้อมูลแบบกระจาย (Distributed Data Processing)	1-2:Batch; 3-4:Online; 5: RT/Multiserver		
3	ประสิทธิภาพของการตอบสนอง (Performance)	0-4:Batch; 4-5:Web; 0-5:RT		
4	ข้อจำกัดของทรัพยากร (Heavily Used Configuration)	2:ทั่วไป; 3-5:Online/RT		
5	ปริมาณการใช้งาน (Transaction Rate)	0-3:Batch; 0-4:Online; 0-5:RT		
6	ปริมาณการบันทึกข้อมูลออนไลน์ (Online Data Entry)	0-1:Batch; 5:Online/RT		
7	ความง่ายในการใช้งาน (End-User Efficiency)	0:Batch; 0-2:Command Line; 3-5:GUI		
8	การแก้ไขข้อมูลออนไลน์ (Online Data Update)	0:Batch; 3:Online		
9	การประมวลผลที่ซับซ้อน (Complex Processing)	%ประมวลผลด้าน Algorithm/Math		
10	การนำกลับมาใช้ใหม่ (Reusability)	0:0%; 1:<10%; 3:>=10%		
11	ความง่ายในการติดตั้ง (Installation Ease)	0:ไม่มีความยาก; 1:เริ่มมีความยาก; 2-3:Install/Conversion; 4-5:Auto Install/Conversion Tool		
12	ความง่ายในการจัดการ (Operational Ease)	0:Backup ทั่วไป; Startup/Backup/Recovery:1-4		
13	ความหลากหลายของสภาพแวดล้อมในการติดตั้ง (Multiple Sites)	0:ติดตั้งที่เดียว; 1:ติดตั้งหลายที่ใน สภาพแวดล้อมเดียวกัน; 2-5:ติดตั้งหลายที่ใน สภาพแวดล้อมต่างกัน		
14	การอำนวยความสะดวกในการเปลี่ยนแปลง (Facilitate Change)	0:รายการของรายงาน; 1-5:ผู้ใช้สามารถเปลี่ยน Data Source หรือรูปแบบของรายงาน		
TDI				
VAF = (TDI * 0.01) + 0.65				

ตารางแสดง แบบฟอร์มการให้คะแนนคุณลักษณะทั่วไปของระบบ

การแปลง Function Point ไปเป็น SLOC (Backfiring Table)

* Languages with updated gearing factors.

+ New languages for which gearing factor data was not previously reported.

Language	QSM SLOC/FP Data			
	Avg	Median	Low	High
ABAP (SAP) *	28	18	16	60
ASP*	51	54	15	69
Assembler *	119	98	25	320
Brio +	14	14	13	16
C *	97	99	39	333
C++ *	50	53	25	80
C# *	54	59	29	70
COBOL *	61	55	23	297
Cognos Impromptu Scripts +	47	42	30	100
Cross System Products (CSP) +	20	18	10	38
Cool:Gen/IEF *	32	24	10	82
Datastage	71	65	31	157
Excel *	209	191	131	315

Focus *	43	45	45	45
FoxPro	36	35	34	38
HTML *	34	40	14	48
J2EE *	46	49	15	67
Java *	53	53	14	134
JavaScript *	47	53	31	63
JCL *	62	48	25	221
LINC II	29	30	22	38
Lotus Notes *	23	21	19	40
Natural *	40	34	34	53
.NET *	57	60	53	60
Oracle *	37	40	17	60
PACBASE *	35	32	22	60
Perl *	24	15	15	60

ที่มา : <http://www.qsm.com/resources/function-point-languages-table>

Do you have any questions?

